«Approved»
By Dean, School of Software Engineering
Associate Prof. Adamova A.I
«01» of 04 2026

Syllabus
Academic Year 2026-2027

1. General information	
Course Code	FP 3222
Course Title	Introduction to Functional Programming
Degree Cycle (Level)/ Major / Relation to curriculum	Bachelor's Degree 6B06101 "Computer Science"
Year, trimester	2nd year, 4th trimester
Language of Instruction:	English
Lecturer(s)/ Instructors/ Instructor Contact Information	Islam Omirzak, PhD Candidate, M.Sc.Ed in Computer Science, Senior lecturer, i.omirzak@astanait.edu.kz, Astana IT University, Expo, C3 block, 3rd floor, office C1.3.354 Office hours: Friday: 18:00-19:00 Zunimova Gulnaz, PhD in Big Data Analytics, Assistant Professor, Astana IT University, Expo, C3 block, 3rd floor, office C1.3.357 Office hours: Wednesday: 18:00-19:00 Pavel Tsoy, M.Sc in Computer Science and Engineering, Teacher, p.tsoy@astanait.edu.kz, Astana IT University, Expo, C3 block, 3rd floor, office C1.3.357 Office hours: Wednesday: 18:00-19:00 Daniil Alexandrov, M.Sc in Computer Science and Engineering, Teacher, D.Alexandrov@astanait.edu.kz, Astana IT University, Expo, C3 block, 3rd floor, office C1.3.357 Office hours: Wednesday: 18:00-19:00
Number of Credits	5
Workload of course components and credits per trimester	

ECTS credits	Contact hours			IASS	SIS	Total hours
	Lectures	Practice sessions	Laboratory work			
5	20	30	-	10	90	150

Prerequisites	Introduction to Programming, Algorithms & Data structures
Post requisites	Advanced programming
2. Goals, objectives and learning outcomes of the course	

1. Course Overview/Description	This course introduces students to functional programming principles using programming languages. Key topics include recursion, immutability, higher-order functions, type systems, monads, lazy evaluation, and error handling.
2. Course Learning Goals	**Understand Functional Programming Concepts:** Develop a deep understanding of core functional programming principles, including recursion, immutability, higher-order functions, and type systems. **Apply Functional Techniques:** Gain practical experience in applying functional programming techniques to solve problems and implement algorithms and data structures. **Utilize Advanced Features:** Explore advanced functional programming features such as monads, lazy evaluation, and error handling to manage complex programming scenarios. **Write Maintainable Code:** Learn to write clear, concise, and maintainable code by leveraging functional programming practices and principles.
3. Course Learning Outcomes	The course aims to equip students with the following key objectives: • **Implement Functional Constructs:** Demonstrate the ability to implement and use fundamental functional programming constructs such as pure functions, recursion, and higher-order functions. • **Work with Data Structures:** Apply functional programming principles to design and implement common data structures, including stacks, queues, and trees. • **Utilize Type Systems:** Understand and use strong, static type systems, including type inference and algebraic data types, to create robust and type-safe code. • **Apply Advanced Features:** Employ advanced functional programming concepts such as monads, functors, and lazy evaluation in practical programming scenarios. • **Handle Errors Effectively:** Implement effective error handling strategies using functional constructs like Maybe and Either. • **Analyze and Optimize Code:** Analyze and optimize functional code for performance and efficiency, including the use of lazy evaluation to handle large or infinite data structures.
4. Methods/forms of teaching	• Individual and group presentations • Practical assessments.
5. Coursework and Grading Scheme	**Assignments:** Assignment 1. Controlling Flow, Converting Types, and Representing Functions in C# Assignment 2. Writing, Debugging, and Testing Functions Assignment 3. Implementing Interfaces and Programming with a Mutable Type Assignment 4. Working with Files, Streams, and Serialization Assignment 5. Composing Functions Using Pipelining Assignment 6. Handling Temporal Coupling Assignment 7. Exception Handling in FP Assignment 8. Extension Methods and LINQ
6. Academic Integrity	Students are expected to uphold the highest standards of academic honesty. All submitted work must be the result of the student's own effort unless group work is explicitly allowed. Plagiarism, cheating on exams, unauthorized collaboration, or copying code from online sources without proper citation are considered violations of academic integrity. Any breach of academic integrity will result in penalties, which may include: • Reduction of grade or a zero for the assignment/exam. • Reporting to the university's disciplinary committee. • Possible failure of the course.

	Maintaining academic integrity is essential for developing genuine knowledge and professional skills
7. Learning resources:	Assigned reading materials and presentations should be read prior to class. Class lectures and discussions will proceed with supplemental and advanced topics, which could be difficult to understand unless students have read the assigned material. Readings are listed in the schedule section. All necessary updates and / or changes to the course will be reflected in the Learning Management System (lms.astanait.edu.kz). Basic Literature: 1. Jon Skeet. C# in Depth / S. Jon. - 4 ed. - Croydon : Manning Publications Co, 2019. - 497 p. 2. Tim Roughgarden: Algorithms Illuminated (Part 3): Greedy Algorithms and Dynamic Programming 3. Tim Roughgarden: Algorithms Illuminated (Part 4): Algorithms for NP-Hard Problems 4. Marius Bancila. Template Metaprogramming with C++ : Learn everything about C++ templates and unlock the power of template metaprogramming / B. Marius. - Birmingham : Packt publishing LTD, 2022. - 455 p 5. Gayle Laakmann McDowell. Cracking the Coding Interview [Текст] : 189 Programming Questions & Solutions / L.M. Gayle. - 6 ed. - Palo Alto : CareerCup, 2019. - 696 p. Supplementary Literature: 1. Yagur, Alex. Functional Programming with C#: Unlock coding brilliance with the power of functional magic. Packt Publishing Ltd, 2024. 2. Sarcar, Vaskaran. "Functional programming overview." Introducing Functional Programming Using C# Leveraging a New Perspective for OOP Developers. Berkeley, CA: Apress, 2023. 3-32. 3. Bandt, Thomas. "Suitability of functional programming for the development of mobile applications: A comparison of F# and C# involving Xamarin." Available at SSRN 3459433 (2019). 4. Price, Mark J. C# 12 and .NET 8–Modern Cross-Platform Development Fundamentals. Packt Publishing Ltd, 2023. 5. Sarcar, Vaskaran. Getting Started with Advanced C#. Apress, 2020.
8. Technology employed:	Online journals, articles, papers, books and internet resources as well as online emulators and online software for simulation.
9. Course Policies	Course Policies for In-Person Instruction: **Presence and Participation** – Attendance is mandatory. Students may not miss more than **30% of class sessions** during the trimester. Exceeding this limit will result in an **F (Fail)** grade (or the requirement to retake the course in summer school). **This rule applies even if absences are supported by medical or other official documents.** – In cases of prolonged health issues or other serious circumstances, students must contact the **university administration** to formally discuss possible academic accommodations for the trimester or academic year. – Students arriving more than 10 minutes late will be marked absent.

	– Class participation is expected and evaluated based on quality of contributions, not just presence. **Deadlines and Late Submissions** – The assignments will be evaluated only if the student has uploaded his/her work to the LMS and defended it on the due date. Without defense, a student cannot get a grade for assignments. You are permitted 3 "late days" for your assignments. • 1 day late: 25% of the maximum allowable marks • 2 days late: 50% of the maximum allowable marks • 3 days late: 75% of the maximum allowable marks – Exceptions may be granted only in extraordinary circumstances with prior approval. **The Honor Code of Students** – No cheating, plagiarism, duplication, falsification of data, or unauthorized collaboration. – Any academic misconduct will result in a zero grade for the work and further disciplinary action. **Code Defense:** Students are required to orally defend and explain their code to the teacher. If a student is unable to explain the solution, the work will not be accepted. **Course Policies for Use of Generative AI** – AI tools may only be used with explicit instructor permission. – Any uncredited or unauthorized AI-generated work will be treated as plagiarism. **Extended Absences** – Absences beyond the allowed limit must be approved by the university administration. – Students are responsible for catching up on missed material. **Late Homework Policy** – At the instructor's discretion **Exam Policy** – Attestation I and II: students must score at least 25% in each to proceed. – The final exam will be a project. **Course Privacy Statement** – Laptops and mobile devices are used only for academic purposes. Misuse will be treated as a disciplinary violation. **Contacting the Instructor** – Students may contact the instructor via Outlook email or Microsoft Teams during working hours **(Mon–Friday, 08:00–18:00)**. – One-on-one consultations are available during office hours (in person or online).
10. Course Schedule	The official course schedule, including lecture and practical session times, is published on the university's system at https://my-du.astanait.edu.kz/app. Students are required to regularly check the system for updates. Any changes to the schedule (such as room changes, rescheduled classes, or instructor substitutions) will be reflected there. The course instructor is not responsible for individually notifying students of schedule updates. Attendance requirements apply according to the updated schedule published in the system.
11. Syllabus Inclusion Statements	Our classroom is a respectful and inclusive environment. The diversity of backgrounds, experiences, and perspectives is valued and encouraged. If you have any accessibility needs, please contact me or the university's disability support services so

	that appropriate accommodations can be arranged.

3.1 Abbreviations

#	Abbreviation	Meaning
1	IASS	Instructor assisted student study - Instructors assist students in achieving success by delivering support, feedback, and resources. This support might take the form of clarifying difficult topics, recommending additional materials, or providing individualized guidance.
2	SIS	Student independent study
3	IW	Individual work
4	GW	Group work
5	PA	Practical assignment
6	LW	Laboratory work
7	MCQ	Multiple choice quiz
8	PBT	Paper-based test

NB: You may introduce abbreviations that are used in the syllabus

3.2 Course Plan

Week No	Course Topic	Lectures (h/w)	Practice sessions (h/w)	Lab. Sessions (h/w)	IASS (h/w)	SIS (h/w)
1	Introduction to FP • Properties of FP • Language Features and Differences with C#	2	3	0	1	9
2	Results-Oriented Programming • Understanding Enumerables • Design Patterns in C#	2	3	0	1	9
3	Pattern Matching • Read-Only Structs • Thread Creation and Management	2	3	0	1	9
4	Custom Enumerations • Parsing Values • Query Adjacent Elements	2	3	0	1	9
5	Higher-Order Functions • Handling nulls • Further improvement	2	3	0	1	9
6	Changeable Code Using Factories • Discriminated Unions • File Handling in C/C++/C#	2	3	0	1	9

7	Add Features Using Wrappers • Map() versus Bind() • Maybe and the Primitives	2	3	0	1	9
8	Currying and Higher-Order Functions • Partial Application • Currying in .NET	2	3	0	1	9
9	Organizing Exceptions • Recursion • Trampolining	2	3	0	1	9
10	Memory Management • Analyzing Memory Leaks • Safe Memory Practices	2	3	0	1	9
	Total hours: 150	**20**	**30**	**0**	**10**	**90**

3.3 Detailed course plan

Week No	Lecture Topics with Learning Resources	Tasks and Activities for Practice Sessions with Recommended Reading Resources
1	Refactor an existing codebase to apply FP principles. • Codebase Assessment • Side-Effect Auditing	Tasks: Understand the principles: immutability, pure functions, and first-class functions. Compare functional vs. object-oriented paradigms. Activities: Write simple pure functions in C#. Refactor an imperative method to a functional style.
2	Implement functions for adding, removing, and searching elements within the data structure. • Immutability • Pure Functions	Tasks: Learn syntax and use cases for lambda expressions. Explore Func<>, Action<>, and custom delegates. Activities: Implement higher-order functions using delegates. Create reusable lambda-based utilities.
3	Develop a multithreaded application that handles multiple client connections simultaneously • Flow Control • Function Composition	Tasks: Understand immutable types and their benefits. Explore readonly, record types, and init properties. Activities: Create immutable models using record. Refactor mutable classes to immutable ones.
4	Analyze and optimize the performance of your application using profiling tools. • Effect Isolation • Architecture	Tasks: Use LINQ for functional-style data transformations. Understand Select, Where, Aggregate, GroupBy. Activities: Build a data pipeline using LINQ. Compare LINQ vs. traditional loops.
5	Implement exception handling and task management to ensure the application's robustness. • Error Handling • Domain Modeling	Tasks: Define and identify pure functions. Understand side effects and how to isolate them. Activities: Refactor impure methods to pure ones. Use dependency injection to manage side effects.

6	Implement and compare two advanced data structures for a given problem • Problem Framing • Candidate Selection	Tasks: Learn how to compose functions. Understand currying and partial application. Activities: Implement function chaining using delegates. Create curried versions of common functions.
7	Write a comparative analysis discussing the trade-offs between the chosen data structures. • Theoretical Complexity • Architectural Analysis	Tasks: Explore alternatives to exceptions: Option, Either, Result. Handle errors using monadic patterns. Activities: Implement a Result<T> type. Refactor exception-heavy code to use functional error handling.
8	Ensure the C library functions are correctly called and handled in the C# application. • Implementation • Interface Design	Tasks: Use record and inheritance to simulate discriminated unions. Apply pattern matching with switch expressions. Activities: Model a domain with union types. Implement pattern matching for domain logic.
9	Propose and develop a full-fledged application using a combination of C and C#. • Empirical Benchmarking • Profiling	Tasks: Design domain models using types and functions. Separate behavior from data. Activities: Build a mini application (e.g., task manager) using functional principles. Apply separation of concerns using pure functions.
10	Covering the design, implementation, and testing phases. • Testing • Migration Strategy	Tasks: Consolidate all concepts into a final project. Review key patterns and anti-patterns. Activities: Build a small functional C# app (e.g., budgeting tool, quiz app). Present code with documentation and design rationale.

3.4 List of assignments for Student Independent Study

№	Assignments (topics) for Independent Study	Hours	Recommended literature and other sources (links)	Form of submission and deadlines
1	2	3	4	5
1	Implement a small application using at least three design patterns	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
2	Create a dynamic data structure using pointers and manual memory management.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
3	Perform a memory analysis using tools like Valgrind to ensure no memory leaks or segmentation faults..	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
4	Implement synchronization mechanisms to avoid race conditions.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version

5	Create an asynchronous application that performs multiple I/O operations without blocking the main thread.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
6	Test the application under different network conditions and report on its performance.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
7	Analyze the time and space complexity of both implementations.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
8	Develop a simple library in C and expose it to a C# application using P/Invoke or COM Interop.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
9	Document the interfacing process and any challenges encountered.	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version
10	File handling, and cross-language integration	9	Reading list, books, internet resources Electronic version	Reading list, books, internet resources Electronic version

3.5 For instructor assisted student study (IASS) schedule and policy follow your course LMS Moodle.

4. Student performance evaluation system for the course.

Period	Assignments*	Number of points	Form of assessment	Total
1st attestation	Assignment 1. Controlling Flow, Converting Types, and Representing Functions in C#	**15**	Practical task, oral defense, in practice session time, quizzes, submission in https://lms.astanait.edu.kz	100
	Assignment 2. Writing, Debugging, and Testing Functions	**15**		
	Assignment 3. Implementing Interfaces and Programming with a Mutable Type	**15**		
	Assignment 4. Working with Files, Streams, and Serialization	**15**		
	Midterm (Practical Exam)	**40**		
2nd attestation	Assignment 5. Composing Functions Using Pipelining	**15**	Practical task, oral defense, in practice session time, quizzes, submission in https://lms.astanait.edu.kz	100
	Assignment 6. Handling Temporal Coupling	**15**		
	Assignment 7. Exception Handling in FP	**15**		
	Assignment 8. Extension Methods and LINQ	**15**		
	Endterm (Practical Exam)	**40**		

Final exam	Presentation of final project	100	Oral defense with presentation, submission in https://lms.astanait.edu.kz	100
Total	**0.3*1 st Att + 0.3 * 2 nd Att + 0,4 * Final**			**100**

Achievement level as per course of curriculum shall be assessed according to the evaluation chart adopted by the academic credit system.

Letter Grade	GPA	Percentage	**Description**
A	4,0	95-100	Excellent
A-	3,67	90-94	
B+	3,33	85-89	Good
B	3,0	80-84	
B-	2,67	75-79	
C+	2,33	70-74	
C	2,0	65-69	Satisfactory
C-	1,67	60-64	
D+	1,33	55-59	
D	1,0	50-54	
FX	0	25-49	Fail
F	0	0-24	

Course Assessment Rubric for Student Performance Evaluation

Learning outcomes	Indicators	Excellent (90-100)	Good (70-89)	Satisfactory (50-69)	Fail 0-49
Describe FP paradigms and use them through development process	Accuracy, clarity, and depth of understanding	Demonstrates outstanding understanding of FP paradigms; Mastery of advanced methods and techniques at a level beyond that explicitly taught	Excellent range and depth of understanding; applies concepts effectively with critical analysis	Demonstrates basic understanding but with noticeable gaps; limited ability to apply concepts independently	Shows poor or no understanding; incoherent explanations or absent work
Formulate original ideas or approaches, demonstrating creativity and problem-solving skills	Originality, logical structure, and feasibility	Produces innovative, well-structured, and feasible solutions and proposals	Produces logical and feasible proposals with some originality	Produces partially feasible proposals; structure weak; originality limited	Proposals incomplete, incoherent, or missing
Collect and analyze data, logical design	Data accuracy, use of tools, interpretation	Excellent, accurate data collection and insightful	Correct data collection and interpretation; shows solid	Some errors in data collection or interpretation; partial grasp of	Data collection and analysis inadequate, incoherent, or

		interpretation; uses advanced tools	understanding	tools	missing
Extensive testing coverage and detailed documentation	Structure, clarity, academic integrity, presentation skills	Outstanding, well-structured, original, academically sound report and presentation	Well-structured, clear, academically sound report and presentation	Report/presentation completed but weak in structure, clarity, or referencing	Report/presentation missing, incoherent, or plagiarized

5. Course assessment methodology

Assessment is conducted continuously throughout the course, comprising 60% continuous assessment throughout the semester and 40% summative assessment during the examination session, totaling 100%. Continuous assessment evaluates students' ongoing performance through attendance, assessment defense, in-class activities, course assignments, and independent study (SIS).

- **IASS (Instructor assisted student study)** - Instructors assist students in achieving success by delivering support, feedback, and resources. This support might take the form of clarifying difficult topics, recommending additional materials, or providing individualized guidance.
- **SIS** – Student engagement in self-directed study on assigned topics and the completion of relevant assignments given in the syllabus.
- **Mid-term and End-term** are a review of the topics covered and an assessment of each student's knowledge. The form of midterm and endterm exams is practical exam.
- **Final assessment** for the course "Introduction to Functional Programming" includes creation of a project with combination of theoretical and practical parts of the course to evaluate the students' academic performance and professional skills. At the completion of the exam, all works must be submitted in the Learning Management System (lms.astanait.edu.kz). No late submissions are allowed in the exam.

6. Lecturer (lecturers) signature

Full name	Job title	Date	Signature
Islam Omirzak	Senior Lecturer	28.08.2026	
Gulnaz Zunimova	Assistant Professor		
Tsoy Pavel	Teacher	28.08.2026	
Daniil Alexandrov	Teacher		

		interpretation; uses advanced tools	understanding	tools	missing
Extensive testing coverage and detailed documentation	Structure, clarity, academic integrity, presentation skills	Outstanding, well-structured, original, academically sound report and presentation	Well-structured, clear, academically sound report and presentation	Report/presentation completed but weak in structure, clarity, or referencing	Report/presentation missing, incoherent, or plagiarized

5. Course assessment methodology

Assessment is conducted continuously throughout the course, comprising 60% continuous assessment throughout the semester and 40% summative assessment during the examination session, totaling 100%. Continuous assessment evaluates students' ongoing performance through attendance, assessment defense, in-class activities, course assignments, and independent study (SIS).

- **IASS (Instructor assisted student study)** - Instructors assist students in achieving success by delivering support, feedback, and resources. This support might take the form of clarifying difficult topics, recommending additional materials, or providing individualized guidance.
- **SIS** – Student engagement in self-directed study on assigned topics and the completion of relevant assignments given in the syllabus.
- **Mid-term and End-term** are a review of the topics covered and an assessment of each student's knowledge. The form of midterm and endterm exams is practical exam.
- **Final assessment** for the course "Introduction to Functional Programming" includes creation of a project with combination of theoretical and practical parts of the course to evaluate the students' academic performance and professional skills. At the completion of the exam, all works must be submitted in the Learning Management System (lms.astanait.edu.kz). No late submissions are allowed in the exam.

6. Lecturer (lecturers) signature

Full name	Job title	Date	Signature
Islam Omirzak	Senior Lecturer	28.08.2026	
Gulnaz Zunimova	Assistant Professor	01.09.2026	
Tsoy Pavel	Teacher	28.08.2026	
Daniil Alexandrov	Teacher	01.09.2026	